BÍBLIA VISUAL

ENGENHARIA, ARQUITETURA E QUALIDADE

Manual prático para modernizar um núcleo de cartões: do domínio e do mainframe até Java, mensageria, PostgreSQL, testes e operação. Cada página traz definição, código aplicável e o que conferir antes de seguir.

MÓDULO 1
Fundamentos
Clean Architecture, microsserviços, DDD, rotina de Dev, QA, Tech Lead, PO e negócio, regras de limite, BMAD, Git e IaC.

MÓDULO 2
Ecossistema
Java com Micronaut, Kafka, SNS, SQS e Transactional Outbox para não perder evento depois do commit.

MÓDULO 3
Qualidade
Pirâmide de testes, k6, JMeter, mutação com Pitest e integração com Testcontainers em Java.

MÓDULO 4
Dados
PostgreSQL, constraints, partição, índices B-Tree e GIN, EXPLAIN ANALYZE, Grafana e New Relic.

MÓDULO 5
Mainframe
COBOL, VSAM, copybook, JCL, mapa de estudo do Vision Plus e migração por Strangler Fig.

MÓDULO 6
Jargão e IA
LLM, RAG, tokens, P0, SLO, post-mortem, RFC e o uso diário de IA por Dev, QA e Tech Lead.

1 **O CASO QUE ATRAVESSA O LIVRO**
Limite de cartão, do pedido até o saldo

Uma compra chega com PAN tokenizado, valor e moeda. O núcleo responde aprovado ou recusado. Se aprovar, o limite disponível desce na hora. No fim do dia a transação posta, entra na fatura e, se o titular não pagar o total, vira rotativo com juros. Os módulos seguintes usam esse mesmo caso.

2 **COMO LER CADA PÁGINA**
O desenho se repete de propósito

- O quadro azul lista o que a página entrega.
- O código é aplicável: Java, SQL, k6, HCL ou COBOL.
- A tabela liga situação, causa e ação.
- O checklist é a porta de saída da página.

MÓDULO 1 · ARQUITETURA

CLEAN ARCHITECTURE NO LIMITE DO CARTÃO

A regra de dependência aponta para dentro. O domínio não conhece Micronaut, Kafka nem PostgreSQL.

O QUE VOCÊ VAI APRENDER

- Separar entidades, casos de uso, adaptadores e detalhes.
- Nomear a regra de dependência e o que ela proíbe.
- Ver um caso de uso Java que não importa framework.
- Saber o que é um porte e o que é um adaptador.

1 QUATRO ANÉIS

Dependência só aponta para o centro

Entidades guardam a regra: limite disponível é limite concedido menos o que já foi reservado e o que já postou. Casos de uso orquestram uma operação, como autorizar compra. Adaptadores traduzem HTTP, SQL e mensagem. Frameworks e drivers ficam no anel de fora. Um import de io.micronaut dentro da entidade quebra a regra.

2 PORTE E ADAPTADOR

O domínio define o contrato

AutorizarCompra.java

```
public interface LimiteRepository {
    Optional<ContaCartao> buscar(ContaId id);
    void salvar(ContaCartao conta);
}

public final class AutorizarCompra {
    private final LimiteRepository limites;
    public AutorizarCompra(LimiteRepository limites) {
        this.limites = limites;
    }
    public Recibo executar(Pedido pedido) {
        ContaCartao conta = limites.buscar(pedido.conta())
            .orElseThrow(() -> new ContaInexistente(pedido.conta()));
        conta.reservar(pedido.valor());
        limites.salvar(conta);
        return Recibo.aprovado(pedido);
    }
}
```

3 O QUE FICA FORA DO DOMÍNIO

Detalhe não entra na regra

- Anotação de controller, entidade JPA e cliente Kafka.
- SQL, DSN de banco e nome de tópico.
- Código de resposta ISO 8583 traduzido na borda, não espalhado no cálculo.
- Relógio e gerador de id entram por porte quando a regra depende deles.

4 TESTE DO MIOLO

Sem banco e sem rede

O teste de AutorizarCompra usa um LimiteRepository em memória. Aprova quando o disponível cobre o valor. Recusa quando não cobre. Esse teste não sobe Docker. A integração com PostgreSQL é outro teste, no módulo de qualidade.

ONDE CADA MUDANÇA ENTRA

Regra no centro, detalhe na borda

Mudança	Anel	Sinal de que foi para o lugar errado
Fórmula do disponível	Entidade	A fórmula aparece só no SQL
Ordem reservar e gravar	Caso de uso	O controller faz os dois passos
JSON do POST	Adaptador	A entidade tem campo chamado payload
Timeout do driver	Detalhe	O caso de uso captura SQLException

CHECKLIST DA REGRA DE DEPENDÊNCIA

- Domínio compila sem o framework.
- Caso de uso recebe portes no construtor.
- SQL mora no adaptador.
- Teste de regra não abre socket.

MÓDULO 1 · MICROSSERVIÇOS

UM SERVIÇO POR CONTEXTO, UM BANCO

Microsserviço é uma fronteira de mudança e de dados. Não é uma pasta a mais no mesmo banco.

O QUE VOCÊ VAI APRENDER

- Decidir o corte pelo fluxo de negócio.
- Comparar chamada síncrona e evento.
- Ver o que quebra quando dois serviços compartilham tabela.
- Aplicar timeout na chamada que não pode segurar a autorização.

1 CORTE DESTE DOMÍNIO

Três serviços, três bancos

- Autorização: decide na hora e reserva limite. Latência baixa.
- Postagem: confirma a transação no fim do ciclo e tira a reserva.
- Fatura: fecha o ciclo, calcula rotativo e emite o boleto.
- Cadastro de produto e plástico não entra no caminho quente da autorização.

2 BANCO POR SERVIÇO

Tabela compartilhada acopla o deploy

Se autorização e fatura leem a mesma tabela limite, a migração de coluna trava os dois. Cada serviço é dono do seu esquema. A fatura não dá SELECT no banco da autorização. Ela consome o evento LimiteAlterado ou pergunta uma API dona do dado. O nome desse desenho é database per service.

3 SÍNCRONO SÓ NO CAMINHO QUE PRECISA DA RESPOSTA

O resto vira evento

fronteiras.txt

```
Autorização  --HTTP, prazo curto--> Risco
Autorização  --evento--> Postagem
Postagem     --evento--> Fatura
Fatura       --não chama--> Autorização no meio da compra
```

4 TIMEOUT É PARTE DO CONTRATO

Sem prazo, a falha vira fila

A chamada de autorização ao risco tem prazo de dezenas de milissegundos e uma resposta padrão se o prazo estourar: recusar ou seguir a política escrita. Sem esse padrão, um risco lento segura todas as compras. O padrão de isolar essa falha se chama bulkhead quando o pool de threads também é separado.

SINCRONO OU EVENTO

A pergunta é se o chamador precisa da resposta agora

Situação	Estilo	Por quê
Compra no caixa	Síncrono	O terminal espera aprovado ou recusado
Montar a fatura	Evento	Pode acontecer depois da compra
Dois serviços, uma tabela	Nenhum dos dois	Acople de esquema, não de mensagem

CHECKLIST DO CORTE

- Cada serviço tem o próprio esquema.
- O caminho quente tem prazo.
- Evento tem dono e nome de negócio.
- Ninguém consulta tabela alheia.

MÓDULO 1 · DDD

DOMÍNIO: CONTA, DINHEIRO E RESERVA

DDD, Domain-Driven Design, coloca o vocabulário do negócio dentro do código. O agregado é a fronteira transacional.

O QUE VOCÊ VAI APRENDER

- Distinguir entidade, objeto de valor e agregado.
- Manter o invariante do limite dentro da conta.
- Publicar um fato de domínio depois da reserva.
- Evitar um agregado que carrega o cliente inteiro.

1 VOCABULÁRIO

O nome no código é o nome na mesa

- Entidade: tem identidade. ContaCartao continua ela mesma se o limite mudar.
- Objeto de valor: Dinheiro e ContaId se comparam pelo conteúdo.
- Agregado: ContaCartao mais as reservas. Uma transação altera o agregado inteiro.
- Repositório: busca e grava o agregado. Não é um DAO genérico de qualquer tabela.
- Evento de domínio: LimiteReservado. Aconteceu. Não é um comando.

2 INVARIANTE NO AGREGADO

Disponível nunca fica negativo

ContaCartao.java

```
public void reservar(Dinheiro valor) {
    if (valor.sinal() != Sinal.POSITIVO) {
        throw new ValorInvalido(valor);
    }
    Dinheiro disponivel = limite.menos(reservado).menos(postado);
    if (valor.maiorQue(disponivel)) {
        throw new LimiteInsuficiente(id, disponivel, valor);
    }
    reservado = reservado.mais(valor);
    eventos.add(new LimiteReservado(id, valor));
}
```

3 O QUE NÃO ENTRA NO AGREGADO

Agregado grande trava linha demais

O nome do titular, o endereço e o histórico de dez anos não viajam na autorização. A conta guarda limite, reservado e postado. O histórico mora no serviço de transações. Ubíquo quer dizer que QA, PO e Dev usam a mesma palavra: reserva não é bloqueio e não é postagem.

4 COMANDO E FATO

Pedir é diferente de ter acontecido

ReservarLimite é um comando: pode falhar. LimiteReservado é um fato: já passou no invariante. O outbox do módulo 2 publica o fato. O consumidor não reabre a conta para decidir de novo a mesma compra.

PEÇA DE DDD

Use o nome certo na revisão

Peça	Pergunta que responde	Exemplo
Entidade	Quem é?	Conta 123
Objeto de valor	Quanto ou qual código?	BRL 50,00
Agregado	O que muda junto?	Conta e suas reservas
Evento	O que já ocorreu?	LimiteReservado

CHECKLIST DO AGREGADO

- Invariante está na entidade.
- Repositório grava o agregado inteiro.
- Evento nasce dentro da operação.
- QA e Dev usam a mesma palavra.

MÓDULO 1 · ROTINA

O DIA DE DEV E DE QA

Entregável é o que outra pessoa consegue usar sem perguntar de novo: PR, teste, evidência e defeito reproduzível.

O QUE VOCÊ VAI APRENDER

- Ver a rotina de um dev Java neste domínio.
- Ver a rotina de um QA Engineer no mesmo fluxo.
- Separar defeito de dúvida de regra.
- Saber o artefato que cada um entrega no mesmo dia.

1 DEV JAVA

Do quadro até o PR verificável

- De manhã: lê o quadro, o alerta da noite e o PR que ficou aberto.
- Antes de codar: escreve o exemplo de aprovado e o de limite insuficiente.
- Durante: altera o agregado, o adaptador e o teste. Não mistura refatoração cega com regra nova.
- Antes do PR: mvnw verify local, descrição do risco, como testar e o que observar em produção.
- Desafio típico: a regra de arredondamento do juros não está no cartão da história.

2 QA ENGINEER

Qualidade não começa no fim da sprint

- De manhã: olha falhas de pipeline, defeitos abertos e a história que entra em refinamento.
- No refinamento: transforma a regra em exemplos. Limite 1000, reserva 200, compra 900, recusa.
- Na história: revisa o teste de unidade do dev e acrescenta integração e contrato.
- Evidência: pedido, resposta, id da conta de teste e consulta do saldo depois.
- Desafio típico: um único usuário de teste compartilhado por todo o time. O módulo 3 trata disso.

3 O QUE CADA UM ENTREGA HOJE

Se não tem artefato, a atividade não terminou

- Dev entrega PR pequeno, teste automático e nota de migração se houve SQL.
- QA entrega exemplos na história, defeito com passos e decisão de risco do release.
- Os dois recusam um PR que só tem teste do caminho feliz da autorização.

RITO DA SEMANA — Presença com entrega

Rito	Dev leva	QA leva
Refinamento	Dúvida técnica e estimativa	Exemplos e buraco de regra
Diário	Bloqueio real	Ambiente ou dado quebrado
Review	PR e risco	Evidência e buraco de teste

CHECKLIST DO DIA

- História tem exemplo numérico.
- PR explica como provar.
- Defeito tem passo e resultado.
- Caminho de recusa tem teste.

MÓDULO 1 · ROTINA

TECH LEAD, PO E NEGÓCIO

Tech Lead protege o sistema. PO ordena o valor. Analista de negócios traduz a regra até ela ficar testável.

O QUE VOCÊ VAI APRENDER

- Ver o que o Tech Lead decide e o que ele não acumula.
- Ver o entregável do Product Owner.
- Ver como o analista de negócios escreve uma regra.
- Separar prioridade de negócio de severidade de produção.

1 TECH LEAD

Arquitetura no cotidiano

- De manhã: incidente, fila de PR e capacidade do time.
- Desenha a fronteira antes de abrir a história grande.
- Escreve ADR, Architecture Decision Record, quando a decisão custa caro para reverter.
- Não é o único revisor. É o desbloqueio e o padrão.
- Entrega: ADR, plano de rollout e meta de SLO do serviço.

2 PRODUCT OWNER

Ordem e corte

- Mantém o objetivo do ciclo: reduzir recusa indevida, por exemplo.
- Corta a história até caber uma regra testável.
- Diz o que fica de fora com a mesma clareza do que entra.
- Aceita a história com o exemplo combinado, não com uma demo só feliz.
- Entrega: backlog ordenado e critério de aceite.

3 ANALISTA DE NEGÓCIOS

Regra sem adjetivo

- Entrevista operação, financeiro e o especialista do legado.
- Escreve a regra com número, exceção e dono.
- Marca o que é regulatório, como IOF, e o que é política da empresa.
- Aponta o copybook ou a tela do legado que ainda manda na regra.
- Entrega: regra versionada e glossário do termo.

4 SEVERIDADE NÃO É PRIORIDADE DE BACKLOG

P0 é produção. P1 de produto é ordem.

P0, P1 e P2 neste livro são severidade de defeito em produção. P0: autorização fora do ar ou limite calculado errado para todos. P1: degradação com contorno. P2: defeito contido. A ordem do backlog do PO usa outro eixo: valor e risco. Misturar os dois números faz um ajuste de texto furar a fila de um P0.

QUEM DECIDE — Uma pergunta, um dono

Pergunta	Dono	Artefato
Esta regra entra agora?	PO	Backlog
Qual serviço é dono do dado?	Tech Lead	ADR
Qual é a fórmula do rotativo?	Analista de negócios	Regra numerada
O teste prova a fórmula?	QA	Exemplo e evidência

CHECKLIST DA DECISÃO

- ADR cabe numa página.
- Regra tem número e exceção.
- Aceite usa o exemplo.
- P0 não entra na fila comum.

MÓDULO 1 · REGRAS

MAPEAR LIMITE SEM DECORAR O LEGADO

Regra de negócio aprendida é exemplo executável. Cartão de crédito é um bom domínio porque cada palavra muda dinheiro.

O QUE VOCÊ VAI APRENDER

- Calcular disponível com reserva, postagem e pagamento.
- Separar autorização, confirmação, fatura e rotativo.
- Usar example mapping para achar buraco.
- Tratar exceção de nome parecido: estorno, chargeback e ajuste.

1 A FÓRMULA DE ESTUDO

Quatro saldos, um disponível

limite.txt

```
disponivel = concedido
           - reservado
           - postado
           + pagamento_nao_faturado

compra 900 com disponivel 800 → recusa
compra 100 com disponivel 800 → reserva 100
postagem da reserva        → reservado cai, postado sobe
pagamento 100 antes do corte → disponivel sobe 100
```

2 EXAMPLE MAPPING

Cartão amarelo é dúvida, não chute

Uma história vira cartões. Azul: a regra. Verde: o exemplo com número. Vermelho: a pergunta sem dono. Amarelo vazio significa que a história ainda não pode ser estimada. Exemplo verde: concedido 1000, reservado 200, postado 300, compra 500, aprova e reservado fica 700. Exemplo vermelho: a compra internacional usa o limite em qual moeda?

3 PALAVRAS QUE O TIME CONFUNDE

Glossário mínimo do cartão

- Autorização reserva. Não gera ainda a linha da fatura.
- Postagem, ou clearing, confirma a transação e move o valor para postado.
- Estorno devolve uma compra do próprio lojista. Chargeback é disputa pela bandeira.
- Rotativo é o saldo não pago depois do vencimento, com juros. Não é o limite.
- IOF, no Brasil, é tributo sobre operação de crédito. A alíquota não mora espalhada em ifs. Mora numa tabela vigente com data.

COMO APRENDER UM DOMÍNIO FECHADO

Fonte, exemplo, prova

Passo	Ação	Pronto quando
Achar a fonte	Operação, legado e norma	Cada regra cita a origem
Numerar	Example mapping	Todo verde tem conta de chegada
Provar	Teste de agregado	O vermelho virou verde ou ficou explícito fora de escopo

CHECKLIST DA REGRA

- Disponível tem fórmula única.
- Reserva e postagem estão separadas.
- Estorno não se chama chargeback.
- Tributo tem vigência.

MÓDULO 1 · BMAD

COMPORTAMENTO, MODELO, API E DADOS

BMAD aqui é o alinhamento dessas quatro camadas. Se uma muda sem as outras, o time entrega um contrato que mente.

O QUE VOCÊ VAI APRENDER

- Amarrar comportamento, modelo, API e dados na mesma história.
- Ver um exemplo completo de reserva de limite.
- Detectar quando a API promete o que o banco não guarda.
- Usar isso no refinamento, não só no desenho.

1 AS QUATRO LETRAS

Uma história preenche as quatro

- B, Behavior: dado concedido 1000 e reservado 0, quando a compra é 400, então aprova e o disponível fica 600.
- M, Model: agregado ContaCartao, invariante do disponível, evento LimiteReservado.
- A, API: POST /autorizacoes com valor e chave de idempotência. 201 aprovado, 422 limite insuficiente.
- D, Data: colunas concedido, reservado, postado e a linha do outbox na mesma transação.

2 O DESENCONTRO TÍPICO

Cada camada conta uma história

A API devolve aprovado, o teste olha só o HTTP 201 e a coluna reservado não muda. O comportamento diz que a reserva existe. O modelo tem o método. O dado não persistiu. BMAD falha. A história só fecha quando o teste integrado lê a coluna e o evento.

3 NO REFINAMENTO

Quadro de uma folha

O analista traz o comportamento numerado. O dev desenha modelo e API. O QA aponta o dado que prova os dois. O PO corta o que não cabe nas quatro células. Se a célula D diz que o legado guarda o limite em centavos inteiros e a API fala em decimal, a conversão entra na história, não vira surpresa na migração.

CÉLULA VAZIA

Não estime por cima do buraco

Vazio	Risco	Ação
Sem comportamento	Cada um testa uma regra	Escrever o exemplo com número
Sem modelo	Regra fica no controller	Nomear agregado e invariante
Sem API	Contrato nasce no código	Fixar status e corpo
Sem dado	O teste não prova persistência	Nomear tabela e coluna

CHECKLIST BMAD

- Comportamento tem número.
- Modelo tem invariante.
- API tem status de recusa.
- Dado prova a reserva.

MÓDULO 1 · ENTREGA

GIT, PIPELINE E INFRAESTRUTURA

IaC, Infrastructure as Code, é a infraestrutura revisada como código. Pipeline é o portão que impede a main de receber o que não passou no teste.

O QUE VOCÊ VAI APRENDER

- Escolher trunk-based para este serviço.
- Ler um pipeline mínimo de GitHub Actions e de GitLab.
- Ver um recurso AWS descrito em Terraform.
- Saber o que não entra no Git: segredo e estado remoto com credencial.

1 BRANCHING

Trunk curto, release protegida

Trunk-based: a branch principal recebe PR pequenos várias vezes ao dia. Feature flag esconde regra incompleta. GitFlow, com develop e release longos, cabe em produto com versão empacotada. Um serviço de autorização contínuo fica melhor no trunk. O PR passa por verificação antes do merge. Isso é o quality gate: o portão automático de teste, análise e política.

2 GITHUB ACTIONS

O mesmo comando que o dev roda

verificar.yml

```yaml
name: verificar
on:
  pull_request:
jobs:
  teste:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./mvnw -B verify
```

3 GITLAB CI

Estágios explícitos

.gitlab-ci.yml

```yaml
stages: [teste, imagem]
unitario:
  stage: teste
  script:
    - ./mvnw -B verify
imagem:
  stage: imagem
  script:
    - docker build -t autorizacao:$CI_COMMIT_SHA .
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

4 TERRAFORM DA FILA

A fila nasce revisada

fila.tf

```
resource "aws_sqs_queue" "autorizacoes" {
  name                       = "cartao-autorizacoes"
  message_retention_seconds  = 345600
  visibility_timeout_seconds = 30
}
# estado no backend remoto
# segredo em variavel, nunca no .tf
```

ONDE MORA CADA COISA

Código sim, segredo não

Item	Lugar	Fora do Git
Pipeline	Repositório	Token de deploy
Terraform	Repositório	Arquivo de estado com senha
Feature flag	Configuração do ambiente	Lista de clientes reais
ADR	docs/adr	Discussão só no chat

CHECKLIST DE ENTREGA

- PR menor que uma regra.
- Pipeline roda mvnw verify.
- IaC da fila está no PR.
- Segredo não aparece no diff.

MÓDULO 2 · JAVA

MICRONAUT NA BORDA, DOMÍNIO NO CENTRO

Micronaut sobe a aplicação, injeta o adaptador e publica o HTTP. A regra de limite continua na classe pura do módulo 1.

O QUE VOCÊ VAI APRENDER

- Ver um controller que só traduz HTTP.
- Ver o adaptador que implementa o porte.
- Usar chave de idempotência na autorização.
- Subir o serviço com o comando do Maven.

1 CONTROLLER

Status é decisão de borda

AutorizacaoController.java

```
@Controller("/autorizacoes")
public class AutorizacaoController {
    private final AutorizarCompra caso;
    public AutorizacaoController(AutorizarCompra caso) {
        this.caso = caso;
    }
    @Post
    public HttpResponse<?> criar(@Body PedidoHttp req) {
        try {
            Recibo recibo = caso.executar(req.paraPedido());
            return HttpResponse.created(recibo);
        } catch (LimiteInsuficiente e) {
            return
HttpResponse.unprocessableEntity().body(e.paraCorpo());
        }
    }
}
```

2 IDEMPOTÊNCIA

O caixa repete o pedido

O terminal reenvia o mesmo POST quando a resposta se perde. A chave Idempotency-Key entra no adaptador. Se a chave já tem recibo gravado, o controller devolve o mesmo recibo e não reserva de novo. Sem essa chave, uma compra vira duas reservas. A chave é dado da borda, consultada antes do caso de uso.

3 ADAPTADOR DO REPOSITÓRIO

Implementa o porte, não a regra

LimiteJdbc.java

```
@Singleton
public class LimiteJdbc implements LimiteRepository {
    private final JdbcOperations jdbc;
    public Optional<ContaCartao> buscar(ContaId id) {
        return jdbc.queryOptional(SQL_BUSCA, id.valor())
            .map(ContaCartao::reidratar);
    }
    public void salvar(ContaCartao conta) {
        jdbc.update(SQL_ATUALIZA, conta.reservado().centavos(),
conta.id().valor());
    }
}
```

4 COMANDOS

O que o dev roda

terminal

```
./mvnw -B test
./mvnw -B mn:run
curl -s -X POST :8080/autorizacoes \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: caixa-88' \
  -d '{"conta":"123","valor":"50.00","moeda":"BRL"}'
```

RESPOSTA HTTP

O domínio não devolve status

Resultado	Status	Corpo
Reserva feita	201	Recibo com id
Limite insuficiente	422	Disponível e valor pedido
Conta inexistente	404	Identificador
Chave repetida	200 ou 201	O mesmo recibo da primeira vez

CHECKLIST DO ADAPTADOR

- Controller sem fórmula.
- Idempotência antes da reserva.
- SQL só no Jdbc.
- 422 é recusa de negócio.

MÓDULO 2 · MENSAGERIA

KAFKA, SNS E SQS

Os três carregam mensagem. Diferem em ordenação, consumo e quem fala com quem. EDA, Event-Driven Architecture, é o desenho em que o fato circula sem o produtor esperar o consumidor.

O QUE VOCÊ VAI APRENDER

- Escolher o sistema pelo contrato de consumo.
- Ver producer Kafka e envio SQS em Java.
- Entender fan-out do SNS.
- Tratar repetição: consumidor idempotente.

1 KAFKA

Log particionado

- A mensagem fica no log até a retenção.
- Partição ordena a chave. A conta é uma boa chave.
- Consumer group divide partições. Cada mensagem do grupo vai a um membro.
- Dá para reler o passado. Serve auditoria e novo consumidor.
- Operação própria ou MSK na AWS.

2 SQS

Fila de trabalho

- Cada mensagem é retirada por um consumidor.
- Standard não garante ordem. FIFO garante ordem por grupo.
- Invisível depois do receive até o visibility timeout.
- Se o worker falha, a mensagem volta. DLQ guarda o que estourou a tentativa.
- Não foi feito para dezenas de consumidores lerem o mesmo fato.

3 SNS

Aviso em leque

- Publica uma vez. Vários assinantes recebem.
- Assinante pode ser SQS, Lambda ou HTTP.
- O padrão SNS para SQS isola o ritmo de cada consumidor.
- Não guarda histórico longo para um consumidor novo reler.
- Bom para o fato LimiteReservado acordar fatura e fraude.

4 KAFKA EM JAVA

Chave é a conta

PublicarKafka.java

```
ProducerRecord<String, String> registro = new ProducerRecord<>(
    "cartao.limite.reservado", contaId, json);
producer.send(registro).get(2, TimeUnit.SECONDS);
```

5 SQS EM JAVA

Uma fila, um trabalho

PublicarSqs.java

```
sqs.sendMessage(SendMessageRequest.builder()
    .queueUrl(url)
    .messageBody(json)
    .messageGroupId(contaId) // só em FIFO
    .build());
```

QUAL USAR NESTE DOMÍNIO

Fato amplo no Kafka ou SNS. Trabalho pontual no SQS

Necessidade	Escolha	Motivo
Vários serviços releem o fato	Kafka	Log retido e novo consumidor
Avisar fatura e fraude já na AWS	SNS para SQS	Fan-out com fila própria
Gerar um PDF de fatura	SQS	Um worker, uma tarefa
Ordem por conta	Kafka ou SQS FIFO	Chave ou message group

CHECKLIST DA MENSAGEM

- Nome do tópico é fato.
- Consumidor aguenta repetição.
- DLQ existe na fila de trabalho.
- Chave da conta preserva ordem.

MÓDULO 2 · CONSISTÊNCIA

TRANSACTIONAL OUTBOX

Gravar o banco e publicar a mensagem em duas chamadas perde o evento ou publica um fato que não existe. O outbox grava os dois na mesma transação.

O QUE VOCÊ VAI APRENDER

- Ver o problema do dual write.
- Criar a tabela outbox.
- Inserir o evento no mesmo commit da reserva.
- Publicar depois, com confirmação, sem mandar duas reservas.

1 O BURACO

Duas escritas, uma falha

Se o serviço faz commit da reserva e o Kafka recusa, o limite baixou e a fatura não fica sabendo. Se publica antes do commit e o commit falha, a fatura recebe uma reserva que não existe. As duas ordens quebram. Dual write é esse par de escritas sem uma transação só.

2 TABELA

O evento é linha

outbox.sql

```
CREATE TABLE outbox (
  id uuid PRIMARY KEY,
  aggregate_id text NOT NULL,
  tipo text NOT NULL,
  payload jsonb NOT NULL,
  criado_em timestamptz NOT NULL DEFAULT now(),
  publicado_em timestamptz
);
CREATE INDEX outbox_pendente
  ON outbox (criado_em)
  WHERE publicado_em IS NULL;
```

3 MESMO COMMIT

Reserva e fato nascem juntos

AutorizarComOutbox.java

```
@Transactional
public Recibo executar(Pedido pedido) {
    ContaCartao conta = limites.buscar(pedido.conta()).orElseThrow();
    conta.reservar(pedido.valor());
    limites.salvar(conta);
    jdbc.update(INSERT_OUTBOX, uuid(), conta.id(),
        "LimiteReservado", conta.ultimoEventoJson());
    return Recibo.aprovado(pedido);
}
```

4 PUBLICADOR

Marca só depois do broker aceitar

Um processo lê as linhas com publicado_em nulo, envia ao Kafka e só então preenche publicado_em. Se cair no meio, a linha continua pendente e o envio se repete. Por isso o consumidor trata LimiteReservado pelo id do evento. Debezium no log do Postgres é a outra forma: lê o WAL e publica, sem poller. O invariante é o mesmo: nada sai antes do commit.

FALHA NO MEIO DO OUTBOX

A linha pendente é a fonte da verdade

Momento	Estado	Ação
Commit ok, publish não	Linha pendente	Publicador tenta de novo
Publish ok, update não	Linha ainda pendente	Reenvio. Consumidor ignora id repetido
Commit falhou	Sem linha	Nada é publicado

CHECKLIST DO OUTBOX

- Insert na mesma transação.
- Índice só das pendentes.
- Publicado só após o ack.
- Consumidor idempotente pelo id.

MÓDULO 2 · FLUXO

DO POST ATÉ A FATURA

Uma compra aprovada atravessa HTTP, agregado, outbox, log e dois consumidores. Cada seta tem um dono.

O QUE VOCÊ VAI APRENDER

- Ler o caminho feliz com os nomes certos.
- Ver o que cada serviço persiste.
- Ver o que não volta para a autorização.
- Nomear o ponto em que a compra já é fato.

1 CAMINHO

Sete passos, um fato

1. Terminal chama POST /autorizacoes com a chave de idempotência.
2. Controller traduz e chama AutorizarCompra.
3. ContaCartao.reservar altera reservado e cria LimiteReservado.
4. A transação grava a conta e a linha do outbox.
5. O publicador entrega o fato no tópico cartao.limite.reservado.
6. Postagem guarda a reserva para casar com o clearing.
7. Fraude lê o mesmo fato. Nenhum dos dois escreve na tabela da autorização.

2 DONO DE CADA DADO

Um escritor por esquema

- Autorização escreve concedido, reservado, postado e outbox.
- Postagem escreve a transação confirmada e o valor que vai a faturar.
- Fatura escreve ciclo, linha de fatura, rotativo e pagamento.
- Leitura cruzada é API ou evento, nunca JOIN entre bancos.

3 QUANDO A COMPRA JÁ VALEU

O commit da reserva

Antes do commit, uma falha desfaz tudo e o terminal pode repetir. Depois do commit, a recusa não existe mais: o caminho é estorno ou expiração da reserva se o clearing não chegar no prazo. Expirar reserva é outro caso de uso, com outro evento, LimiteLiberado.

EVENTO

Nome no passado, carga mínima

Evento	Quem publica	Quem consome
LimiteReservado	Autorização	Postagem e fraude
TransacaoPostada	Postagem	Fatura e autorização
FaturaFechada	Fatura	Cobrança e notificação
LimiteLiberado	Autorização	Postagem

CHECKLIST DO FLUXO

- Um escritor por tabela.
- Fato só depois do commit.
- Consumidor não devolve comando.
- Expiração tem evento próprio.

MÓDULO 3 · ESTRATÉGIA

PIRÂMIDE E GESTÃO DA QUALIDADE

A pirâmide coloca a maior parte da prova em teste rápido e deixa o teste de ponta a ponta curto. Gestão de qualidade é o risco coberto, não a quantidade de casos.

O QUE VOCÊ VAI APRENDER

- Distribuir unitário, integração, contrato e ponta a ponta.
- Escolher a ferramenta Java de cada camada.
- Tratar web e mobile sem duplicar a regra.
- Decidir o release pelo buraco, não pelo percentual solto.

1 CAMADAS

Quanto sobe, mais lento e mais frágil

- Unitário: agregado e caso de uso. JUnit 5 e AssertJ. Milhares, segundos.
- Integração: JDBC, outbox, SQS. Testcontainers. Dezenas, minutos.
- Contrato: o JSON que o consumidor espera. Pact JVM ou Spring Cloud Contract. Sem Node.
- Ponta a ponta: uma compra de verdade num ambiente. Poucos. Selenium ou o canal do app.
- Estático: SpotBugs, Error Prone, checagem de migração SQL no pipeline.

2 WEB, BACKEND E APP

A regra mora uma vez

O cálculo do disponível não é retestado na UI com cinquenta combinações. A UI prova que o botão mostra a recusa que a API já devolveu. O app prova o mesmo contrato, mais offline e permissão. O backend prova a fórmula. Shift-left é trazer essa prova para o refinamento e o PR, não deixar a descoberta para a véspera do go-live.

3 GESTÃO DO CICLO

Risco nomeado

- Cada história nasce com os exemplos BMAD e o nível da pirâmide que os cobre.
- Defeito de produção vira teste no nível em que a falha podia ter sido vista.
- Quality gate do PR: teste, mutação mínima do pacote alterado e sem segredo no diff.
- Release: SLO da autorização estável, migração ensaiada, rollback escrito.

ONDE PROVAR

Não suba de camada sem necessidade

Afirmação	Camada	Ferramenta
900 recusa quando disponível é 800	Unitário	JUnit
A coluna reservado mudou	Integração	Testcontainers
O JSON tem o campo recibo	Contrato	Pact JVM
O caixa vê a recusa	Ponta a ponta	Um cenário

CHECKLIST DA ESTRATÉGIA

- Fórmula tem teste unitário.
- Persistência tem integração.
- Contrato cobre a recusa.
- E2E não repete a tabela verdade.

MÓDULO 3 · PERFORMANCE

K6 E JMETER COM UMA CONTA

Teste de carga mede o caminho. Teste de stress passa do limite de propósito. Uma conta de teste só serve se o cenário não depender de saldo compartilhado.

O QUE VOCÊ VAI APRENDER

- Escrever um script k6 com limiar.
- Isolar a conta única com chave e valor pequeno.
- Rodar JMeter sem a interface gráfica.
- Separar metrica do gerador e metrica do serviço.

1 K6

Limiar que falha o pipeline

carga.js

```
import http from 'k6/http';
import { check } from 'k6';
export const options = {
  vus: 30,
  duration: '2m',
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<300'],
  },
};
export default function () {
  const res = http.post(__ENV.BASE_URL + '/autorizacoes',
    JSON.stringify({ conta: __ENV.CONTA, valor: '1.00',
      moeda: 'BRL', chave: __VU + '-' + __ITER }),
    { headers: { 'Content-Type': 'application/json' } });
  check(res, { 'contrato': r => r.status === 201 || r.status === 422 });
}
```

2 UMA CONTA SÓ

Não dispute o mesmo saldo

- Cenário de leitura, GET do disponível, pode usar a conta à vontade.
- Cenário de escrita usa valor 1.00 e chave VU-ITER. A idempotência evita reserva dupla do mesmo iter.
- O limite da conta de laboratório é alto o bastante para o teste, ou o teste aceita 422 como contrato válido.
- Não afirme o saldo final: trinta VUs em paralelo não têm um saldo determinístico.
- Limpeza é um job depois, não um DELETE no meio da medição.
- Se precisar de saldo exato, use uma conta por VU criada na setup. Uma conta só não dá essa garantia.

3 JMETER EM LINHA DE COMANDO

GUI só desenha o plano

jmeter.sh

```
jmeter -n -t autorizacao.jmx -l resultado.jtl \
  -JbaseUrl=https://lab.exemplo -Jconta=conta-lab \
  -Jusuarios=30 -Jduracao=120
```

4 COMO LER

O gerador também satura

p95 alto com CPU do serviço baixa pode ser o gerador ou a rede. Olhe saturação dos dois lados. Stress é subir VUs até o erro aparecer e anotar o ponto. Soak é segurar a carga moderada por horas para achar vazamento de pool. Os dois usam o mesmo script. Muda o perfil, não a regra de negócio.

PERFIL — Um script, três perguntas

Perfil	Forma	Pergunta
Carga	VUs previstos, p95	Aguenta o pico do dia?
Stress	Sobe até quebrar	Onde e como quebra?
Soak	Carga média, horas	Pool ou memória escorre?

CHECKLIST DA CARGA

- Limiar de erro e p95.
- Chave por iteração.
- 422 conta como contrato.
- Métrica do serviço junto.

MÓDULO 3 · MUTAÇÃO

PITEST OLHA O TESTE, NÃO A LINHA

Cobertura diz que a linha rodou. Mutação muda o código e exige que o teste falhe. Se o teste continua verde, a suíte não segurou a regra.

O QUE VOCÊ VAI APRENDER

- Entender o que é um mutante.
- Ver um mutante clássico do limite.
- Configurar o Pitest no Maven.
- Não perseguir 100 por cento cego.

1 POR DENTRO

Matar o mutante

O Pitest copia uma classe, aplica um operador e roda os testes que cobrem essa classe. Operador de condição troca maiorQue por maiorOuIgual. Se o teste de limite 800 contra compra 800 não existe, os dois comportamentos passam e o mutante sobrevive. Mutante morto significa que algum teste ficou vermelho. Mutante sobrevivente é buraco. Timeout e mutante equivalente são inconclusivos, não vitória.

2 O MUTANTE QUE IMPORTA

A fronteira do disponível

mutante.java

```
// original
if (valor.maiorQue(disponivel)) throw new LimiteInsuficiente(...);
// mutante: condição invertida ou trocada por ≥
if (!valor.maiorQue(disponivel)) throw new LimiteInsuficiente(...);
// o teste que mata: compra == disponivel aprova
// e compra == disponivel + 0.01 recusa
```

3 MAVEN

Portão no pacote que mudou

pom.xml

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.17.4</version>
  <configuration>
    <targetClasses><param>com.exemplo.limite.*</param></targetClasses>
    <targetTests><param>com.exemplo.limite.*</param></targetTests>
    <mutationThreshold>75</mutationThreshold>
  </configuration>
</plugin>
./mvnw -B pitest:mutationCoverage
```

RESULTADO DO PITEST

Aja no sobrevivente de regra

Estado	Significado	Ação
Killed	Um teste ficou vermelho	Siga
Survived	Nenhum teste percebeu	Teste na fronteira
No coverage	Nenhum teste executou	Teste ou código morto
Timed out	O mutante não terminou	Olhe laço e mock

CHECKLIST DA MUTAÇÃO

- Fronteira igual ao limite testada.
- Limiar no pacote de domínio.
- Sobrevivente vira teste.
- Não mutar DTO sem regra.

MÓDULO 3 · INTEGRAÇÃO

TESTCONTAINERS SEM NODE

Testcontainers sobe Postgres e LocalStack no Docker para o teste JUnit. O teste prova SQL e fila de verdade e acaba quando a JVM acaba.

O QUE VOCÊ VAI APRENDER

- Subir PostgreSQL 16 no teste.
- Gravar reserva e linha de outbox.
- Subir SQS no LocalStack.
- Manter o teste independente da ordem.

1 POSTGRES

Esquema real, dado do teste

ReservaIT.java

```
@Testcontainers
class ReservaIT {
  @Container
  static PostgreSQLContainer<?> pg =
      new PostgreSQLContainer<>("postgres:16-alpine");
  @Test
  void gravaReservaEOutbox() {
    Executar.jdbc(pg).executar(pedidoDe("40.00"));
    long pendentes = contar(pg,
      "select count(*) from outbox where publicado_em is null");
    assertThat(pendentes).isEqualTo(1);
  }
}
```

2 SQS LOCAL

AWS de mentira, SDK de verdade

FilaIT.java

```
@Container
static LocalStackContainer localstack =
    new
LocalStackContainer(DockerImageName.parse("localstack/localstack:3.8"))
        .withServices(Service.SQS);
SqsClient cliente = SqsClient.builder()
    .endpointOverride(localstack.getEndpoint())
    .credentialsProvider(StaticCredentialsProvider.create(
        AwsBasicCredentials.create(localstack.getAccessKey(),
localstack.getSecretKey())))
    .region(Region.of(localstack.getRegion()))
    .build();
```

3 HIGIENE

O teste não herda sujeira

- Cada teste cria a conta que usa, com id aleatório.
- Não há conta fixa compartilhada com o teste de carga.
- A asserção olha a linha pelo id do teste, não count(*) da tabela inteira, se outros testes dividem o container.
- O container é estático na classe para não pagar o boot a cada método. A transação do teste desfaz o que puder.
- CI precisa de Docker. Sem Docker o teste de integração não finge que passou.

O QUE ESTE TESTE NÃO FAZ

Integração não é carga nem UI

Fora de escopo	Onde prova	Por quê
30 usuários paralelos	k6	Container local não é o ambiente
Pixel do app	E2E do canal	Aqui não há tela
Mutante da fórmula	Pitest	A fórmula já tem teste unitário

CHECKLIST DA INTEGRAÇÃO

- Imagem com versão fixa.
- Dado criado pelo teste.
- Outbox e reserva juntos.
- CI tem Docker.

MÓDULO 4 · POSTGRESQL

TABELA, REGRA E PARTIÇÃO

O banco guarda o invariante que o aplicativo também guarda. Constraint não substitui o agregado. Impede a linha impossível que escapou de um caminho.

O QUE VOCÊ VAI APRENDER

- Criar a conta com tipos e checks.
- Impedir disponível negativo por coluna gerada ou check.
- Particionar transação por tempo.
- Migrar com script versionado.

1 CONTA

Dinheiro em centavos inteiros

V001__conta.sql

```
CREATE TABLE conta (
  id uuid PRIMARY KEY,
  concedido_centavos bigint NOT NULL CHECK (concedido_centavos ⩾ 0),
  reservado_centavos bigint NOT NULL CHECK (reservado_centavos ⩾ 0),
  postado_centavos bigint NOT NULL CHECK (postado_centavos ⩾ 0),
  CHECK (reservado_centavos + postado_centavos ⩽ concedido_centavos),
  versao bigint NOT NULL DEFAULT 0
);
```

2 POR QUE CENTAVOS

Numeric é válido. Inteiro evita surpresa

numeric(15,2) também serve e documenta a escala. Inteiro em centavos evita uma parte dos arredondamentos no meio do Java. Os dois são aceitáveis se a escolha for uma só, da borda até a fatura. Misturar double é defeito: 0.1 não é exato em binário. O check da soma impede uma reserva que o código esqueceu de validar.

3 PARTIÇÃO DA TRANSAÇÃO

Corte por mês

V002__transacao.sql

```
CREATE TABLE transacao (
  id bigint NOT NULL,
  conta_id uuid NOT NULL,
  centavos bigint NOT NULL CHECK (centavos <> 0),
  ocorrida_em timestamptz NOT NULL,
  PRIMARY KEY (id, ocorrida_em)
) PARTITION BY RANGE (ocorrida_em);

CREATE TABLE transacao_2026_11 PARTITION OF transacao
  FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');
```

CONSTRAINT

O banco recusa a linha

Regra	Mecanismo	Erro se violar
Id único	PRIMARY KEY	Chave duplicada
Valor presente	NOT NULL	Nulo
Soma dentro do limite	CHECK	Check violation
Mês certo	PARTITION	Nenhuma partição

CHECKLIST DO ESQUEMA

- Sem double para dinheiro.
- Check do limite na tabela.
- Partição inclui a chave.
- Script tem versão.

MÓDULO 4 · ÍNDICES

B-TREE, GIN E O PLANO

Índice acelera leitura e encarece escrita. EXPLAIN ANALYZE executa a consulta e mostra o custo real. EXPLAIN sem ANALYZE só estima.

O QUE VOCÊ VAI APRENDER

- Escolher B-Tree para igualdade e faixa.
- Escolher GIN para jsonb e lista.
- Saber quando não criar índice.
- Ler Seq Scan versus Index Scan no plano.

1 TIPOS

O acesso manda no tipo

- B-Tree é o padrão. Serve =, <, >, BETWEEN e ORDER BY alinhado.
- GIN, Generalized Inverted Index, serve jsonb, array e busca de termos. A escrita é mais cara.
- Hash só faz igualdade. Desde o PostgreSQL 10 ele tem WAL, mas quase nunca ganha de um B-Tree. Não é a primeira escolha.
- Índice parcial, com WHERE, cabe nas linhas do outbox ainda não publicadas.

2 QUANDO NÃO CRIAR

Índice ocioso também é custo

- Coluna boolean sozinha, com metade da tabela em cada lado. O planejador prefere ler a tabela.
- Tabela pequena que cabe em poucas páginas.
- Coluna que muda o tempo todo e nenhuma consulta filtra por ela.
- Índice duplicado da chave primária.
- Ordem das colunas invertida: (ocorrida_em, conta_id) não atende WHERE conta_id = ? sozinho com a mesma força de (conta_id, ocorrida_em).

3 PLANO

Olhe linhas de verdade e buffers

plano.sql

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, centavos
FROM transacao
WHERE conta_id = '6b1e2a7c-1d24-4a3e-9f0a-0c2d9a1b77e2'
  AND ocorrida_em ⩾ now() - interval '30 days';
-- Index Scan usando (conta_id, ocorrida_em): caminho esperado
-- Seq Scan com milhões de linhas: falta índice ou a estatística está velha
ANALYZE transacao;
```

SINTOMA NO PLANO

Uma causa comum

O que você vê	Causa comum	Ação
Seq Scan na tabela grande	Filtro sem índice útil	Índice na ordem do WHERE
Index Scan e mesmo assim lento	Volta muita linha à tabela	Índice cobre as colunas do SELECT
Estimativa 10 e realidade 1 milhão	Estatística velha	ANALYZE
Bitmap na jsonb	Consulta de containment	GIN no jsonb

CHECKLIST DO ÍNDICE

- WHERE dita a ordem das colunas.
- Parcial no outbox pendente.
- ANALYZE depois da carga.
- Índice novo justificado pelo plano.

MÓDULO 4 · OBSERVABILIDADE

SINAIS DE OURO, GRAFANA E NEW RELIC

Os quatro golden signals são latência, tráfego, erros e saturação. Log explica um caso. Métrica mostra a tendência. Trace atravessa autorização, banco e fila.

O QUE VOCÊ VAI APRENDER

- Definir os quatro sinais na autorização.
- Ler um PromQL e um NRQL.
- Propagar o trace id até a mensagem.
- Distinguir SLI, SLO e SLA com número.

1 OS QUATRO SINAIS

Painel mínimo do serviço

- Latência: p95 do POST /autorizacoes. Meta de estudo: abaixo de 300 ms.
- Tráfego: requisições por segundo, separadas por aprovado e recusado.
- Erros: taxa de HTTP 5xx. 422 é negócio, não erro de plataforma.
- Saturação: pool de conexões, CPU, lag do consumidor e fila do outbox pendente.

2 CONSULTAS

Grafana lê Prometheus. New Relic lê NRQL

sinais.txt

```
# PromQL, p95 em 5 minutos
histogram_quantile(0.95,
  sum by (le) (rate(http_server_requests_seconds_bucket{
    uri="/autorizacoes"}[5m])))

-- NRQL
SELECT percentile(duration, 95)
FROM Transaction
WHERE name = 'POST /autorizacoes'
SINCE 1 hour ago
```

3 TRACE DISTRIBUÍDO

Um id atravessa o fato

O adaptador HTTP abre o span. O JDBC e o publicador do outbox são spans filhos. O trace id vai no cabeçalho da mensagem. O consumidor da fatura continua o trace. Sem isso, o p95 alto não diz se o tempo está no banco, no Kafka ou na regra. Log sem esse id obriga a cruzar horário na mão.

4 SLI, SLO, SLA

Medida, meta e contrato

SLI, Service Level Indicator, é a medida: porcentagem de autorizações com HTTP diferente de 5xx e p95 dentro do prazo, numa janela de 30 dias. SLO, Service Level Objective, é a meta interna: 99,9 por cento. SLA, Service Level Agreement, é o contrato com consequência se a meta externa quebrar. Alerta queima o orçamento de erro do SLO, não o susto de um minuto isolado.

ONDE OLHAR PRIMEIRO

Sinal, depois log

Sintoma	Sinal	Próximo passo
Compra lenta	Latência p95	Trace do p95, não a média
Fila crescendo	Saturação / lag	Consumidor e visibility timeout
Cliente vê falha	Erros 5xx	Log com o trace id
Recusa subiu	Tráfego por código	Regra de negócio, não saturação

CHECKLIST DO PAINEL

- 422 fora da taxa de erro.
- p95 com trace.
- Lag da fila no painel.
- SLO escrito com janela.

MÓDULO 5 · MAINFRAME

COBOL, VSAM, COPYBOOK E JCL

O legado de cartão ainda decide limite em batch e em programa online. Ler o layout é a primeira ferramenta da migração.

O QUE VOCÊ VAI APRENDER

- Ler as divisões de um programa COBOL curto.
- Entender VSAM KSDS como arquivo indexado.
- Ver um copybook de saldo.
- Ler um JCL que executa o programa.

1 PROGRAMA

A regra cabe no PROCEDURE

AUTORIZA.cbl

```
IDENTIFICATION DIVISION.
PROGRAM-ID. AUTORIZA.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-DISPONIVEL PIC S9(9)V99 COMP-3.
PROCEDURE DIVISION.
    COMPUTE WS-DISPONIVEL = WS-LIMITE - WS-RESERVADO - WS-POSTADO
    IF WS-VALOR > WS-DISPONIVEL
        MOVE '51' TO WS-RESPOSTA
    ELSE
        ADD WS-VALOR TO WS-RESERVADO
        MOVE '00' TO WS-RESPOSTA
    END-IF.
    STOP RUN.
```

2 PEÇAS

O que cada nome é

- IDENTIFICATION, ENVIRONMENT, DATA e PROCEDURE são as quatro divisões clássicas.
- PIC descreve o campo. S9(9)V99 é sinal, nove dígitos e duas casas. COMP-3 é decimal compactado.
- COPY inclui um copybook, o layout compartilhado. Mudar o copybook muda todo programa que o copia.
- VSAM KSDS, Key-Sequenced Data Set, é arquivo indexado pela chave. A conta é a chave. Não é uma tabela SQL, mas o acesso por chave se parece com um índice.
- JCL, Job Control Language, não é a regra. É o script que aloca arquivo e chama o programa.

3 COPYBOOK

O contrato binário do legado

SALDO.cpy

```
01 SALDO-CONTA.
   05 CONTA-ID        PIC X(16).
   05 LIMITE          PIC S9(9)V99 COMP-3.
   05 RESERVADO       PIC S9(9)V99 COMP-3.
   05 POSTADO         PIC S9(9)V99 COMP-3.
```

4 JCL

Job, step, programa, arquivo

AUTORIZ.jcl

```
//AUTORIZ JOB (ACCT),'CARTAO',CLASS=A,MSGCLASS=X
//STEP1   EXEC PGM=AUTORIZA
//STEPLIB DD DSN=CARTAO.LOADLIB,DISP=SHR
//CONTAS  DD DSN=CARTAO.CONTAS.KSDS,DISP=SHR
```

TRADUÇÃO DE LEITURA

Não reescreva antes de nomear

Legado	No Java deste livro	Cuidado
COPYBOOK	DTO de borda e teste de layout	Campo packed muda o byte
KSDS por conta	Tabela conta com PK	A chave pode ter preenchimento
JOB noturno	Worker com outbox	Horário de corte é regra
Código 51	422 limite insuficiente	00 e 51 são ISO 8583

CHECKLIST DO LEGADO

- Copybook versionado.
- PIC entendido em centavos.
- JCL lido até o DD.
- Código 51 mapeado.

MÓDULO 5 · VISION PLUS

NÚCLEO DE CARTÃO: AUTORIZAR E FATURAR

Vision Plus é um núcleo de cartão usado em muitos emissores. Os nomes abaixo são o mapa de estudo usado em modernização. Não são o manual interno do fabricante.

O QUE VOCÊ VAI APRENDER

- Situar CMS, FAS, TRAMS, ITS e CTA no fluxo.
- Separar autorização online de postagem e fatura.
- Ver onde nascem limite, juros e transação.
- Saber o que o Java novo precisa substituir primeiro.

1 MAPA DE ESTUDO

Cinco nomes, cinco responsabilidades usuais

- CMS, gestão de cartão e conta: plástico, produto, limite concedido e status.
- FAS, autorização financeira: responde a compra em tempo real com o código 00 ou 51.
- TRAMS, transações monetárias: posta o clearing, estorno e ajuste no saldo.
- ITS, borda de intercâmbio: fala com a bandeira e traduz a mensagem.
- CTA, conta e movimento do titular: base que fatura e cobrança consultam no fim do ciclo.

2 ORDEM DO DINHEIRO

Cada etapa tem horário

A compra passa no ITS, o FAS reserva no limite do CMS e responde. Horas ou dias depois o clearing chega, o TRAMS posta e a reserva sai. No corte, o CTA fecha a fatura. O que não foi pago vira rotativo: saldo vezes a taxa do período, mais tributo vigente. Juros não são calculados na autorização do caixa.

3 O QUE DÓI NA MIGRAÇÃO

Estado espalhado

- O limite concedido mora no CMS e o reservado pode estar no FAS. Os dois precisam de uma fórmula só.
- O batch noturno do TRAMS e o online do FAS alteram saldo. A ordem entre eles é regra.
- Copybook da mensagem de autorização é o contrato com a bandeira. Mudou o byte, mudou o emissor.
- O primeiro corte útil costuma ser a leitura: um serviço Java responde disponível enquanto o legado ainda grava. Escrever nos dois ao mesmo tempo, sem dono, recria o dual write.

PERGUNTA DE NEGÓCIO

Qual módulo de estudo responde

Pergunta	Módulo de estudo	Dado
Qual é o limite do produto?	CMS	Concedido
A compra de agora passa?	FAS	Disponível e código
O clearing entrou?	TRAMS	Postado e reserva liberada
Quanto é a fatura?	CTA	Ciclo, rotativo, tributo

CHECKLIST DO MAPA

- Fórmula única de disponível.
- Corte do batch escrito.
- Copybook da bandeira isolado.
- Juros fora da autorização.

MÓDULO 5 · MIGRAÇÃO

STRANGLER FIG E ANTICORRUPÇÃO

Strangler Fig substitui o legado por fora, um fluxo de cada vez, até o programa antigo perder o tráfego. ACL, Anti-Corruption Layer, impede o copybook de virar o modelo Java.

O QUE VOCÊ VAI APRENDER

- Cortar a autorização primeiro, com o legado ainda dono do resto.
- Traduzir copybook na ACL, não no agregado.
- Sincronizar limite sem dual write eterno.
- Definir o dia em que o FAS deixa de decidir.

1 FIGUEIRA ESTRANGULADORA

Uma rota por vez

- Uma fachada recebe a mensagem da bandeira.
- A rota de autorização de compra nacional vai para o Java.
- Internacional, ajuste e estorno continuam no FAS até terem exemplos.
- A fachada é o único lugar que conhece os dois mundos.
- Quando a rota antiga fica sem tráfego, o programa COBOL correspondente sai.

2 ACL

Byte do legado não entra no domínio

AclAutorizacao.java

```
public final class AclAutorizacao {
    public Pedido traduzir(byte[] iso) {
        String conta = ebcdic(iso, 0, 16).trim();
        long centavos = packed(iso, 16, 6);
        return new Pedido(ContaId.de(conta),
Dinheiro.centavos(centavos));
    }
    public byte[] responder(Recibo recibo) {
        return recibo.aprovado() ? codigo("00") : codigo("51");
    }
}
```

3 DADOS NO MEIO DO CAMINHO

Aurora PostgreSQL é o destino, não o atalho

Enquanto o CMS ainda altera o concedido, um fluxo de mudança copia o concedido para a conta Java. A autorização nova é dona só do reservado que ela mesma criou. Postagem continua no TRAMS e devolve TransacaoPostada. Quando fatura e postagem também migram, o legado deixa de escrever. Aurora PostgreSQL recebe o esquema do módulo 4. O outbox publica para quem ainda escuta. Não há período infinito em que os dois lados reservam a mesma compra.

FASE — Dono da reserva

Fase	Quem reserva	Legado
0	FAS	Tudo
1	Java na compra nacional	Resto e concedido
2	Java também na postagem	Fatura e produto
3	Java	Somente leitura até desligar

CHECKLIST DA MIGRAÇÃO

- Uma rota por vez.
- ACL sem regra de limite.
- Um dono da reserva.
- Plano de voltar a rota.

MÓDULO 6 · GLOSSÁRIO

IA E GESTÃO, SEM FRASE SOLTA

Jargão só ajuda se a frase seguinte disser o que fazer. Esta página fixa o sentido usado no restante da bíblia.

O QUE VOCÊ VAI APRENDER

- Explicar LLM, token, contexto, prompt, RAG, fine-tuning e agente.
- Explicar gate, severidade, rollout, SLO e post-mortem.
- Usar cada termo na autorização de cartão.
- Não trocar SLA por SLO.

1 IA PARA QUEM CONSTRÓI O SISTEMA

O modelo prevê texto. Não consulta o banco sozinho

- LLM, Large Language Model: modelo que continua texto. Não é uma base de regras do limite.
- Token: pedaço de texto que o modelo conta. Um copybook grande gasta a janela.
- Context window: o quanto cabe numa conversa. O que ficou de fora não foi lido.
- Prompt: a instrução e o contexto que você entrega. Exemplo e restrição mudam a resposta.
- RAG, Retrieval-Augmented Generation: buscar um trecho seu, como o ADR, e só então pedir a resposta.
- Fine-tuning: treinar o modelo com dados seus. Caro e desnecessário para explicar um PR.
- Agente: o modelo chama ferramentas em sequência, por exemplo ler o teste e rodar o Maven. A ferramenta é que altera arquivo.

2 GESTÃO TÉCNICA

A palavra muda a fila

- Quality gate: portão automático. O merge espera o pipeline.
- P0: produção quebrada ou dinheiro errado para muitos. P1: degradado com contorno. P2: contido.
- Rollout: ligar a rota nova aos poucos. Rollback: devolver a rota ao FAS.
- Go-live: o momento em que a rota passa a valer para compra real.
- SLI mede, SLO é a meta interna, SLA é o contrato.
- Post-mortem: relato do incidente sem caça ao culpado, com a ação que vira teste.
- RFC, Request for Comments: proposta de mudança aberta ao comentário antes do ADR.
- Tech debt: atalho com juros. Anote o custo, não use a palavra como enfeite.
- Shift-left: prova e regra cedo, no refinamento e no PR.

3 FRASE PRONTA NESTE DOMÍNIO

Termo e consequência

Um rollout de 5 por cento da autorização nacional tem rollback para o FAS em um comando da fachada. O SLO é 99,9 por cento de respostas sem 5xx em 30 dias. Se o p95 estoura, o post-mortem liga o trace e cria o teste que faltou. Um RFC precede a troca do dono da reserva. Tech debt é a ACL que ainda traduz um copybook que ninguém mais emite.

NÃO CONFUNDIR

Pares que o incidente mistura

Dito	Sentido certo	Não é
SLA de 99,9	Contrato	Não é o gráfico de ontem
P0 no backlog	Produção parada	Não é história importante
Fine-tuning	Treino do modelo	Não é colar o ADR no chat
Agente	Modelo com ferramenta	Não é um dev a mais sem revisão

CHECKLIST DO TERMO

- SLO tem número e janela.
- P0 tem dono agora.
- RFC antes do corte grande.
- RAG cita o documento.

MÓDULO 6 · PRÁTICA

IA NO DIA DE DEV, QA E TECH LEAD

A IA acelera rascunho, explicação e teste. Quem assina o PR continua responsável pelo invariante do limite e pelo segredo que não pode ir no prompt.

O QUE VOCÊ VAI APRENDER

- Pedir um teste de fronteira com o exemplo BMAD.
- Pedir explicação de copybook sem colar produção.
- Revisar o diff como se a IA fosse um par júnior.
- Recusar prompt que inclui senha, PAN ou token de cliente.

1 DEV

Rascunho em volta da regra

- Cole o agregado e peça testes nas fronteiras: igual ao disponível, um centavo acima, valor zero.
- Peça a tradução do PIC para um teste de layout, com um exemplo numérico seu.
- Peça um refator que extraia o adaptador. Leia o diff. Rode ./mvnw -B test.
- Não peça para a ferramenta ignorar o quality gate.
- Se o modelo inventar um método do Micronaut, confira na documentação da versão do projeto.

2 QA

Exemplo e buraco

- Entregue a regra numerada e peça os casos de example mapping que faltam.
- Peça o script k6 a partir do contrato, com 422 aceito e chave por iteração.
- Peça para apontar afirmação sem prova no PR. Confira se a afirmação existe mesmo.
- Dados de produção, PAN e senha não entram no prompt. Use conta fictícia.

3 TECH LEAD

Síntese com fonte

- RAG nos ADRs antes de pedir um desenho novo. A resposta deve citar o ADR.
- Peça o rascunho do post-mortem a partir da linha do tempo. Você escreve a ação.
- Peça a lista de rotas ainda no FAS. Confira no código da fachada.
- Use a IA para achar tech debt nomeado, não para aprovar o próprio PR.

4 PROMPT QUE FUNCIONA

Contexto, restrição, prova

prompt.txt

```
Contexto: ContaCartao.reservar lança
LimiteInsuficiente se valor > disponivel.
Tarefa: testes JUnit 5 dessas fronteiras:
igual, um centavo acima, zero, negativo.
Restrição: não altere produção.
Prova: o comando ./mvnw -B test tem que passar.
```

O QUE REVISAR SEMPRE

A ferramenta não assina

Saída	Risco	Prova
Teste novo	Afirma o contrário da regra	Ler o exemplo numerado
SQL	Apaga constraint	Rodar no Testcontainers
Explicação de legado	Inventa módulo	Abrir o copybook
Pipeline	Guarda segredo	Ler o diff do YAML

CHECKLIST DO USO

- Sem PAN e sem senha.
- Teste roda de verdade.
- ADR citado quando houver.
- Diff lido por uma pessoa.